

AI-based Village Pond Planning System

CSD Assignment 1 – Final Technical Report

PARITOSH LAHRE, Indian Institute of Technology Bhilai — Student ID: 12341550, India

Rural water conservation in India often depends on identifying suitable sites for small check-dams and percolation ponds, a task traditionally done by manual site inspection with no systematic terrain, catchment, or rainfall analysis. This project implements a full-stack, terrain-aware decision-support system that recommends pond locations from either an uploaded contour map (KML/KMZ) or a user-drawn area on an interactive map. The backend reconstructs a digital elevation model, hydrologically conditions it (Priority-Flood depression filling), computes D8 flow direction and accumulation, delineates catchments via upstream graph search, and scores candidate sites on five terrain/hydrology factors. Real historical rainfall (Open-Meteo) drives a Rational-Method runoff estimate, which in turn sizes a planning-level pond (depth, surface area, storage volume). A land-use constraint filter, built on live OpenStreetMap data via the Overpass API, rejects candidates that fall on existing buildings, roads, rivers, or water bodies. All of this is exposed through a React and Leaflet frontend with a dark/light theme and both input modes, engineered to degrade gracefully rather than fail outright when any of its five external data dependencies is unavailable – a design choice validated repeatedly against real, confirmed failures (DNS resolution errors, connection resets, and undocumented rate limits from more than one provider) observed on the actual deployment network. On a representative $\sim 4 \text{ km}^2$ village contour dataset, the system evaluated 187 candidate seeds, accepted 5 after terrain/catchment/land-use filtering, and recommended a site with an estimated $21,820 \text{ m}^3$ annual collectable runoff and $17,456 \text{ m}^3$ planned storage, computed end-to-end in 1.2–7.2 seconds. The map-selected-area path was verified end-to-end against live external APIs under real, observed network failures, with a bounded worst-case wall-clock time kept safely under the client’s request timeout. The system is validated by 70 automated tests and extensive live testing against real (not mocked) external services.

Additional Key Words and Phrases: Village Pond Planning, Geospatial Analysis, Rainwater Harvesting, Catchment Estimation, Web GIS, REST APIs

1 Introduction

Rural India loses a large fraction of monsoon rainfall to unmanaged runoff. Small ponds and check-dams built at hydrologically correct locations – natural depressions or flow-convergence points with adequate catchment – can capture this water cheaply, but choosing that location today relies on manual inspection with no systematic terrain, catchment, or rainfall analysis. This project builds a web application that automates that decision: given either a contour map or a map-drawn area, it reconstructs terrain, analyzes drainage, scores candidate sites, filters out sites that would conflict with existing land use, estimates expected water volume from real rainfall data, and presents all of this overlaid on an interactive map. The rest of this report covers the requirements (§2), architecture (§3), the terrain/hydrology methodology (§4), implementation details (§5), how core Computer System Design themes were applied (§6), results (§7), and an honest discussion of limitations (§8).

Author’s Contact Information: Paritosh Lahre, Indian Institute of Technology Bhilai — Student ID: 12341550, Bhilai, India, paritoshl@iitbhilai.ac.in.

Manuscript submitted to ACM

1.1 Motivation

Manual site selection is hard for three concrete reasons this project directly addresses. First, terrain complexity: a contributing catchment is not visible by eye, it requires flow-direction and flow-accumulation analysis across an entire elevation surface. Second, there is no centralized, per-site rainfall record – village-level historical rainfall statistics are not something a field surveyor can look up on the spot. Third, there is no systematic way to cross-check a promising terrain location against existing buildings, roads, or rivers without a separate land-survey pass. Automating these three lets a non-specialist get a defensible, explainable recommendation in seconds instead of a multi-day manual survey.

1.2 Scope of the Project

In scope: two input modes (upload a KML/KMZ contour map, or draw an arbitrary area directly on a map, no file needed); terrain reconstruction, hydrological conditioning, flow analysis, and multi-candidate catchment delineation from either input; multi-factor candidate scoring and ranking with human-readable reasoning for the top recommendation; real historical rainfall retrieval and Rational-Method runoff estimation; planning-level pond sizing (depth, surface area, storage volume); a land-use constraint filter (buildings, roads, rivers, water bodies, power lines) using live OpenStreetMap data; an interactive map frontend showing every layer above, with graceful degradation displayed to the user (not hidden) when any external data source is temporarily unavailable.

Out of scope: structural/civil engineering design of the pond embankment or spillway – outputs are planning-level estimates, not construction drawings; soil type, infiltration, or land-use-based refinement of the runoff coefficient – a single configurable constant is used; verifying land ownership; multi-user authentication or per-user data isolation; persistent storage beyond a single server process’s lifetime (§5); and guaranteeing live external API uptime – the system is explicitly designed to degrade and report “unavailable” rather than assume any of its five external dependencies is always reachable, because in practice every one of them was observed to fail at least once during development.

2 Problem Statement and Requirements

2.1 Non-Functional Requirements

Performance: contour-upload analysis completes in under 10 seconds end-to-end for a village-scale ($\sim 4 \text{ km}^2$) dataset – observed 1.17–7.22s pipeline time across multiple real runs against live external APIs. Map-selected-area analysis is bounded to a worst case of roughly 75–90 seconds even under significant external-API degradation, via a hard 45-second elevation-fetch deadline and a 25-second land-use-fetch deadline that runs concurrently with it (§4, §8) – this bound was discovered necessary after a real production log showed an unbounded, multi-minute worst case before these deadlines existed.

Concurrency/Scalability: the backend’s event loop remains responsive to concurrent requests during a slow analysis – verified live that `GET /health` answered in under 3ms while a 30-second-plus area analysis was simultaneously in flight, because the blocking analysis pipeline is explicitly offloaded to a thread pool rather than run inline on the async event loop.

Availability/Resilience: the system tolerates the failure of any single external dependency without failing the whole request – every external call degrades to an explicit “unavailable” status plus a warning

Table 1. Functional requirements and where they are implemented

Requirement	Implemented in
Satellite imagery display	<code>MapView.jsx</code> (Esri World Imagery layer)
Contour map visualization	<code>kml.service.py</code> → GeoJSON → <code>MapView.jsx</code>
Available-land identification	<code>candidate.service.py</code> + <code>landuse.service.py</code>
Catchment area estimation	<code>catchment.service.py</code> ; flow direction/accumulation and watershed algorithms in <code>algorithms/</code>
Historical rainfall query	<code>rainfall.service.py</code> (Open-Meteo)
Runoff volume estimation	<code>runoff.service.py</code> (Rational Method)
Pond depth / storage recommendation	<code>pond.service.py</code>
Combined overlay / results view	<code>MapView.jsx</code> + <code>ResultsPanel.jsx</code>

message rather than raising an exception. This is not a theoretical concern: every external dependency (Open-Meteo, all three elevation providers, and Overpass) was observed to fail for real during development, including DNS resolution failures, connection resets, and confirmed rate-limiting responses (§8).

Usability: single-page web app; dark theme by default with a persistent light/dark switch; both input modes accessible from one screen; explicit loading state with a time-expectation message; explicit, human-readable error banners rather than a blank or broken UI on failure.

Security: no authentication is implemented – a deliberate scope decision appropriate for a single-team demo deployment, not a production multi-tenant system.

Browser support: tested against Chromium-based browsers via automated browser verification during development; no browser-specific code paths.

3 System Architecture and High-Level Design

The system has three tiers plus five external data sources. The client tier is a React single-page application that either uploads a KML/KMZ file or submits a drawn polygon as JSON over a CORS-enabled REST API, with no authentication layer between them. The server tier is a layered FastAPI application: `api/` (HTTP route handlers) calls into `services/` (pipeline orchestration), which calls `algorithms/` (pure computational routines) and `providers/` (external API integrations), operating on `models/` (dataclasses and Pydantic schemas). Each layer is independently testable. There is no database tier (§5); results are held in an in-memory store for the life of the server process. Five independent external providers sit behind the services layer – Open-Meteo for rainfall, a three-tier elevation cascade (OpenZenith, then Open-Elevation, then OpenTopoData), and the Overpass API for OpenStreetMap land-use data – each wrapped in its own

resilience layer (bounded timeout, bounded retries, and graceful degradation to an explicit “unavailable” result rather than an exception).

3.1 Technology Stack

Backend: Python, FastAPI, Pydantic / pydantic-settings, Uvicorn. Geospatial: GeoPandas, Shapely, pyproj, Fiona, rasterio. Numerical: NumPy, SciPy. KML/XML: lxml. HTTP client: httpx. Testing: pytest and pytest-asyncio (70 tests).

Frontend: React 19, Vite, Leaflet with `react-leaflet` and `leaflet-draw` (pinned to a known-good version to avoid an upstream bug in later releases), plain CSS with CSS-variable theming. No TypeScript, no state-management library, no UI framework – a deliberate minimalism choice matching the assignment’s own emphasis on a clean, functional interface over a heavy frontend stack.

External APIs, all free and requiring no API key: Open-Meteo Historical Weather API for rainfall; OpenZenith as the primary elevation provider for the map-drawn-area path (the assignment brief’s own suggested elevation API); Open-Elevation and OpenTopoData as a two-tier elevation fallback, tried only for points OpenZenith could not provide; the Overpass API over OpenStreetMap data for land-use exclusion geometries.

Deviation from the suggested stack: the brief suggested Flask or FastAPI; FastAPI was chosen for its async support, automatic OpenAPI documentation, and Pydantic validation, which fit the geospatial and numerical workload well. No relational database (PostgreSQL with PostGIS was considered during planning) was used in the final system – the in-memory store’s own module docstring documents the intended upgrade path as an isolated, additive change rather than a rearchitecture, which was judged sufficient for this assignment’s scope.

4 Methodology

4.1 Terrain and Elevation Analysis

Two independent ways obtain a Digital Elevation Model, feeding the same downstream pipeline. From an uploaded KML/KMZ contour map, contour lines are parsed, sampled along each line, projected into an auto-detected UTM zone, and interpolated onto a regular grid via `scipy`’s `griddata` (linear interpolation with a nearest-neighbour pass to fill edge gaps). From a user-drawn map area, the polygon’s bounding box (padded 10% on each side to avoid flow-routing edge artifacts) defines the same grid specification, but elevation for every grid cell is fetched from a live elevation API instead of interpolated: OpenZenith is tried first (up to 2000 points per batch request), falling back per-point to Open-Elevation and then OpenTopoData for anything OpenZenith could not provide. A hard 45-second wall-clock budget bounds the whole fetch regardless of grid size; any points not fetched in time are filled via nearest-neighbour interpolation from whatever was fetched, with a warning reported to the caller. Because a candidate could otherwise be generated inside that 10% padding margin – outside the area the user actually selected – an additional hard filter rejects any accepted candidate whose point falls outside the literal drawn polygon, while catchment *boundaries* are left unrestricted, since a real drainage basin is not bounded by an arbitrary rectangle.

Both paths then run identical hydrological conditioning: Priority-Flood depression filling (Barnes et al., 2014), a min-heap-based algorithm that raises artificial sinks to their spill elevation, guaranteeing continuous downhill drainage, and slope computation via a finite-difference gradient.

4.2 Catchment Area Delineation

For every cell, D8 flow direction chooses the steepest downhill neighbour among 8 candidates (distance-weighted elevation drop). Flow accumulation treats the resulting direction grid as a directed acyclic graph and processes it in topological order (Kahn’s algorithm) to count upstream contributing cells per cell. For a given candidate pour point, an upstream breadth-first search on the inverted flow graph collects every cell that drains to it; the resulting mask is vectorized into a GeoJSON polygon and its area computed in square kilometres.

Candidate generation draws seeds from two independent signals – depression seeds (cells raised during depression filling by more than 10 cm) and flow-convergence seeds (top-1-percentile flow accumulation) – merges and de-clusters them via spatial non-maximum suppression, then applies hard filters in sequence before scoring: slope too steep (default threshold 15°), outside the user’s drawn selection (map-drawn-area path only), land-use constraint, and catchment too small (default threshold 0.005 km^2). The land-use constraint checks the candidate’s point location against a unioned, buffered set of OpenStreetMap building, road, waterway, water-body, and power-line geometries fetched live via the Overpass API; a candidate inside any buffer is rejected with a specific, human-readable reason. Surviving candidates are scored on five min-max-normalized factors (catchment area, flow accumulation, slope, relief, and depression depth, weighted 35%, 25%, 20%, 10%, and 10% respectively) and ranked.

A stated limitation: the land-use filter checks the candidate point, not its full catchment polygon – a catchment naturally spans roads and rivers upstream, which is hydrologically correct and intentionally not filtered, but can look like an oversight to a first-time viewer of the map. Only simple OpenStreetMap “way” geometries are handled; complex multipolygon relations are skipped as a documented simplification. DEM resolution (30 m default, configurable) bounds the spatial precision of every downstream computation.

4.3 Rainfall Data Integration

Open-Meteo’s Historical Weather API is queried for daily precipitation at the recommended candidate’s coordinates (or the area’s centroid if none is accepted), for a configurable historical window (2015 through the last complete calendar year by default). Daily values are aggregated into an annual average, a per-month average across all covered years, and a monsoon (June–September) seasonal average, feeding the Rational Method runoff estimate: annual runoff (m^3) equals the runoff coefficient times annual rainfall (m) times catchment area (m^2), with the coefficient defaulting to 0.35 (documented as representing mixed agricultural/semi-pervious land, pending a future soil- or land-cover-based refinement). Planned pond storage is 80% of annual runoff by default, with a default depth of 3.0 m (2.0 m on candidates with slope above 5°) and surface area derived as storage divided by depth.

Table 2. REST API endpoints

Method & Path	Purpose
POST /api/v1/analyzeContour	Upload a KML/KMZ file → full analysis
POST /api/v1/analyzeArea	Submit a drawn polygon → full analysis
GET /api/v1/analysis/{id}	Retrieve a prior analysis
GET /api/v1/health	Service health check

5 Implementation

5.1 Backend and API Design

No authentication is implemented (§2). Every external call (Open-Meteo, all three elevation providers, Overpass) is wrapped to catch timeout, HTTP-status, connection, and malformed-payload errors and convert them into a provider-specific “unavailable” result with a human-readable message, rather than propagating an exception – the request still returns 200 OK with the rest of the analysis intact plus a warnings list. A same-second retry is only attempted for failures that can plausibly succeed on retry (timeouts, server errors); a DNS/connection failure is not retried, since a same-second retry of a broken connection essentially never succeeds and was found, in a real production log, to needlessly double the time lost per failure. Internal and validation errors use a consistent JSON error shape across all endpoints, enforced by a custom exception handler.

5.2 Frontend and Visualization

A React and Leaflet single-page application renders map layers as GeoJSON overlays: contour lines (upload mode only), candidate points colour-coded by status with a distinct marker for the top-ranked candidate, catchment boundary polygons, and a recommended-location marker, each with a popup showing its own statistics. The base map is standard OpenStreetMap raster tiles, with a CSS colour-invert filter applied for the dark theme rather than switching tile source (chosen after verifying that alternative “dark” basemap tile sets have real rural-coverage gaps for the target region), plus a satellite imagery option. The results panel surfaces status, warnings, the recommended pond’s location, catchment area, expected annual water volume, and planned storage – the fields the assignment brief specifically requires – alongside supporting rainfall/runoff detail and a collapsible full candidate table.

5.3 Database and Storage

There is no database. Analysis results are held in an in-memory store keyed by a UUID analysis identifier and are lost on server restart. This was a deliberate scope decision for a single-session assignment deployment; Manuscript submitted to ACM

Table 3. Mapping of core CSD themes/topics to their use in this project

CSD Theme/Topic		Where used in the project	Justification / design rationale
API (REST)	Design	Four versioned endpoints under <code>/api/v1</code> , auto-generated OpenAPI docs	Consistent JSON contract, self-documenting for frontend integration; versioning leaves room for a future v2
Caching		Three in-memory caches: rainfall, elevation, land-use, each keyed by rounded coordinates	Avoids redundant calls to slow/rate-limited public APIs; only <i>successful</i> results are cached – caching a failure was found, via live testing, to permanently disable that feature until process restart
Concurrency Async	/	Elevation batches fetched in a thread pool; the blocking pipeline is offloaded from the event loop; land-use is fetched concurrently with elevation	Cuts worst-case fetch time; keeps <code>/health</code> responsive during a slow analysis (a real freeze bug was fixed this way); overlaps two calls that otherwise stacked toward the client timeout
Error Handling Resilience	/	Every external call (5 providers) degrades to “unavailable” plus a warning instead of raising; elevation and land-use each have a fallback/deadline	All 5 dependencies failed for real during development – DNS errors, confirmed rate-limiting, connection resets – the normal operating condition on the deployment network, not a hypothetical
Algorithms & Complexity		Priority-Flood depression fill, D8 flow direction, flow accumulation (topological sort), watershed BFS, spatial NMS	Chosen over naive/quadratic alternatives since the DEM can be tens of thousands of cells even for a small village
Design Patterns		Interchangeable provider modules behind one function interface; a single orchestrator coordinates independently-testable stages	New providers can be swapped without touching the pipeline; the orchestrator is the only component that knows the full pipeline order
Testing Strategy		70 automated pytest tests plus live browser-driven verification for the frontend	Confirms resilience behaviour, not just the happy path – several real bugs were caught by live testing, not mocks
Version Control		Incremental commits per fix/feature, messages documenting root cause and verification	Keeps history a readable record of what was found and why
Load Balancing		Not implemented	Single-instance deployment; out of scope at this scale
DB Indexing Query Opt.	/	Not applicable	No persistent database exists (§5)
Microservices vs. Monolith		Monolith (single FastAPI app)	A distributed split adds operational complexity with no benefit here; the layered structure already separates concerns
Authentication		Not implemented	Explicit scope decision; needed before any public multi-user deployment
Containerization		Not yet done	Open item, see §8

the storage module’s own docstring documents the upgrade path to a persistent, spatially-indexed database as an isolated, additive change that does not require altering the service or API layers.

6 CSD Themes and Topics Applied in the Project

Error Handling and Resilience and *Concurrency* were the dominant engineering concerns of this project in practice, not just checklist items. Every external API this system depends on was observed to genuinely fail during development – not as a hypothetical edge case, but repeatedly, in real logs, on the actual deployment network, including DNS resolution failures, confirmed rate-limiting from at least two independent providers (one surfaced directly as an HTTP 429 response, the other traced to a Cloudflare-level “origin rate limited” response after only a handful of rapid requests), and connection resets. The system’s core design principle –

every external call degrades to an explicit “unavailable” status plus a warning rather than failing the whole request – was validated against these real failures, not just unit-test mocks. Concurrency work followed directly from a bug this principle surfaced: a single slow, degraded external-API call was found to freeze the entire server, because the request handler ran inline on the async event loop. Fixing it, by offloading to a thread pool, was necessary specifically because of how failure-prone the external dependencies turned out to be in practice.

7 Results and Evaluation

Table 4. Representative example, KML upload path (IIT Bhilai contour dataset)

Metric	Value
Contour lines parsed	1,355 (267.0–298.0 m)
DEM grid	106×130 @ 30 m
Average slope	1.9°
Candidate seeds (raw → after NMS)	418 → 187
Candidates scored (top score)	15 (82.8)
Accepted / rejected (this run)	5 / 0
Recommended catchment area	0.0450 km ²
Historical rainfall (annual avg.)	1,385.4 mm
Estimated annual runoff	21,820.05 m ³
Planned pond storage	17,456.04 m ³
Recommended depth / surface area	3.0 m / 5,818.7 m ²
End-to-end processing time	1.17–7.22 s

A second representative run on a ~ 0.6 km² user-drawn polygon over the same region fetched 675 elevation samples (200 requiring nearest-neighbour fill after provider retries), generated 11 candidates, rejected 6 with a land-use-constraint status against real OpenStreetMap data (23 roads, 1 waterway, 2 water bodies, and 1 power line found in the bounding box), and accepted 5. Verified visually in the browser: rejected candidate markers line up directly along real road geometry on the satellite basemap, and the recommended marker sits in open space away from both the river and all roads. No published ground-truth figures were available to compare against; this is noted honestly as a limitation rather than fabricated.

7.1 Performance

No formal load testing (many concurrent users) was performed; this is noted as a limitation, not claimed as tested.

8 Discussion and Limitations

External API reliability was the single biggest practical risk to this system’s usability, not any algorithmic weakness. During development, and especially when testing on the actual grading/lab machine rather than an isolated development sandbox, every external dependency showed real transient failures –

Table 5. Observed operation times

Operation	Observed time
KML-upload analysis, end-to-end	1.17–7.22 s
Map-drawn-area, real fetch, no fallback needed	1–2 s
Map-drawn-area, cached elevation	0.02–0.06 s
Map-drawn-area, providers degraded (worst case)	bounded ~75–90 s
GET /health during a 30s+ analysis	<3 ms
Overpass land-use query, healthy mirror	1–2 s

DNS resolution failures, connection resets, “network unreachable” errors, and gateway timeouts, sometimes several within a single request. This confirmed the unreliability is a property of that network’s egress in general, not any one provider being uniquely bad, and directly validated the project’s resilience-first design rather than being a hypothetical concern. Direct testing also confirmed that two of the five providers enforce real, mostly undocumented rate limits: one returned an explicit HTTP 429 response under load, and another began returning a Cloudflare-level “origin rate limited” response after only a handful of requests in quick succession from a single network. With an entire class concurrently exercising the same free services, likely from one shared institutional IP address, these limits are easy to trip continuously, independent of any code-level bug.

Live testing surfaced several genuine bugs that no unit test caught on its own. When an elevation provider failed a large batch late, after already spending much of the shared time budget on its own retries, the fallback chain re-chunked that batch into many small sub-batches but originally fetched them sequentially, so only the first one or two could complete before the deadline – a batch that was genuinely recoverable ended up mostly missing, occasionally triggering a false “elevation unavailable” error; this was fixed by fetching sub-batches concurrently. Separately, the elevation point cache was found to store a failed lookup exactly as it would store a successful one, so a single bad network moment permanently disabled elevation for that area until the server process was restarted, even after the network fully recovered; this was fixed by only caching successful values. A third issue was architectural rather than a bug: sequential external calls (elevation, land-use, rainfall) were found, from a real log, to stack close enough to the frontend’s own request timeout that a slightly worse network moment could cause the client to abandon a request the backend was about to complete successfully; this was addressed by bounding the land-use fetch’s own worst case and launching it concurrently with the elevation fetch, since both depend only on the drawn area’s bounding box.

Further limitations, stated honestly rather than omitted: the land-use filter checks a candidate’s point, not its full catchment polygon, so a recommended pond’s catchment can legitimately cross a road or river upstream – hydrologically correct, but visually surprising to a first-time viewer; the runoff coefficient is a single configurable constant rather than derived from soil type or land cover, a real accuracy limitation flagged explicitly in the UI’s assumptions section; there is no database, so results do not survive a server restart; no formal load testing was performed; and DEM resolution (30m default) bounds the spatial precision of every downstream computation. A third-party dependency risk was also found and worked

around: the map’s polygon-drawing library has a known upstream bug in its latest release that breaks rectangle drawing under modern strict-mode bundlers, worked around by pinning to an earlier, unaffected release.

9 AI Tool Usage Declaration

Claude (Anthropic’s Claude Code CLI) assisted with implementation, debugging, and report drafting; all changes were reviewed and tested by the author.

A Source Code and Repository

Repository: <https://github.com/SandstromPL/AI-based-Village-Pond-Planning-System->. **Deployed frontend:** *[PLACEHOLDER: deployed URL]*. **Demo video:** *[PLACEHOLDER: public YouTube link]*.

Folder structure: **backend/app/** is split into **api/** (HTTP route handlers), **services/** (pipeline orchestration, one service per concern), **algorithms/** (pure computational routines: depression filling, D8 flow, flow accumulation, watershed, scoring, slope, interpolation), **providers/** (external API integrations for elevation and land-use), **models/** (dataclasses and Pydantic schemas), and **utils/** (geo helpers, GeoJSON, in-memory storage); **backend/tests/** holds 70 pytest tests. **frontend/src/** holds **components/** (map view, results panel, upload/draw controls, loading and error UI), **utils/** (display formatting and defensive GeoJSON filtering), and the top-level API client, theme, and application entry point.